



IITMO

o DevOps

- Ops
 - Dev (python)
 - DevOps
 - Облачные технологии, 10+ лет
 - Финтех, FinOps[, MLops?]
-
- VTB/T1/Иннотех облако
 - ITMO: облачные технологии, контейнеризация, DevOps-практики, Ops+SRE

Что такое DevOps?



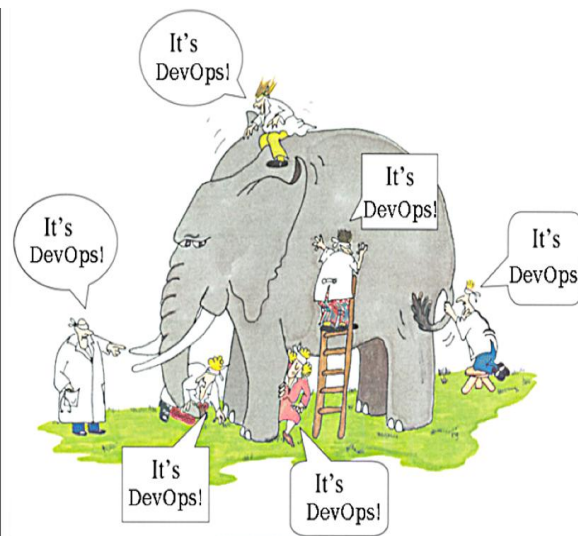


CALMS framework:

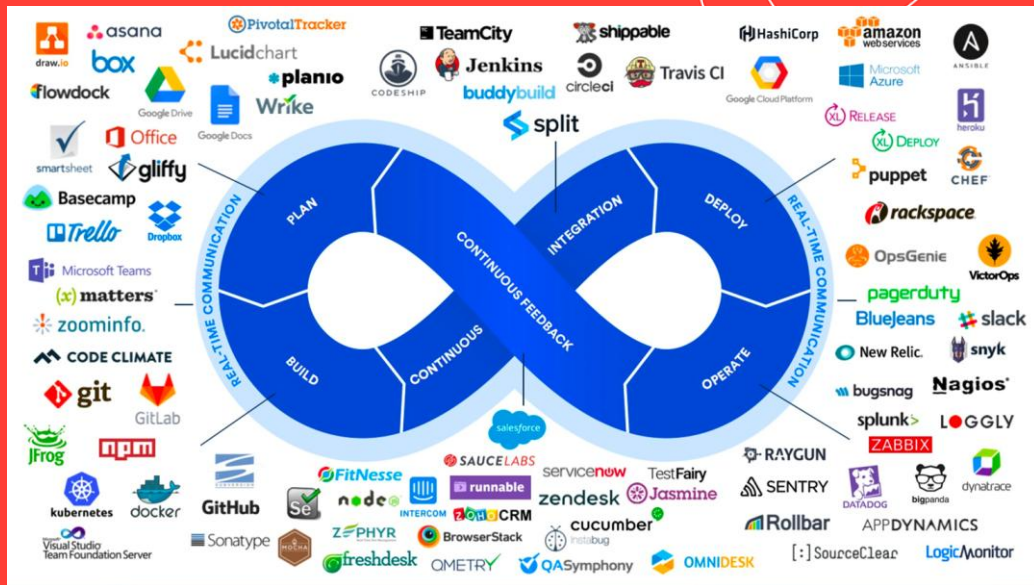
- **C**ulture - культура сотрудничества
- **A**utomation - автоматизация процессов
- **L**ean - оптимизация потока создания ценности
- **M**easurement - измерение всего
- **S**haring - обмен знаниями

Three Ways:

1. Flow (поток) - от идеи к production быстро
2. Feedback (обратная связь) - мониторинг, логи, метрики
3. Continuous Learning - эксперименты, постмортемы



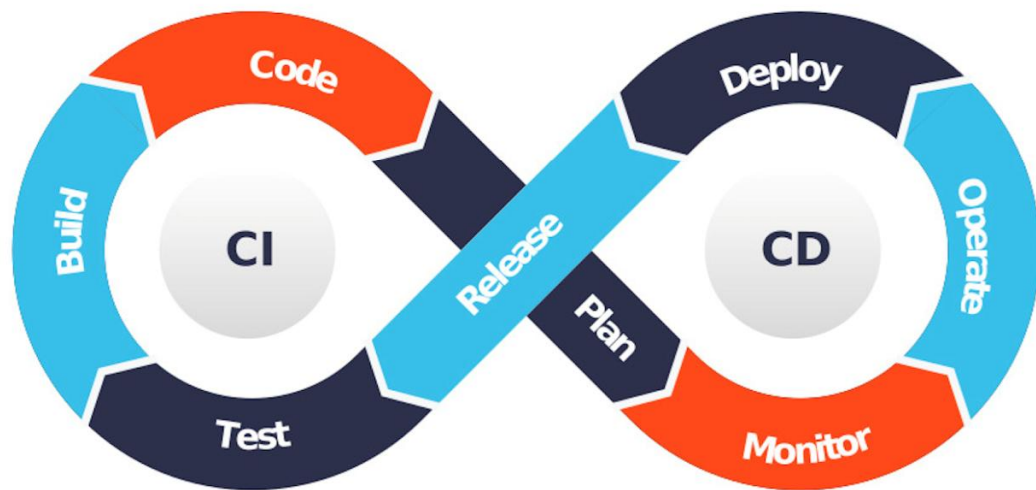
Что в себя включает?



- *Непрерывная интеграция и доставка*
- *Гибкая разработка ПО*
- Управление версиями
- *Инфраструктура как код*
- Мониторинг
- Observability
- и так далее

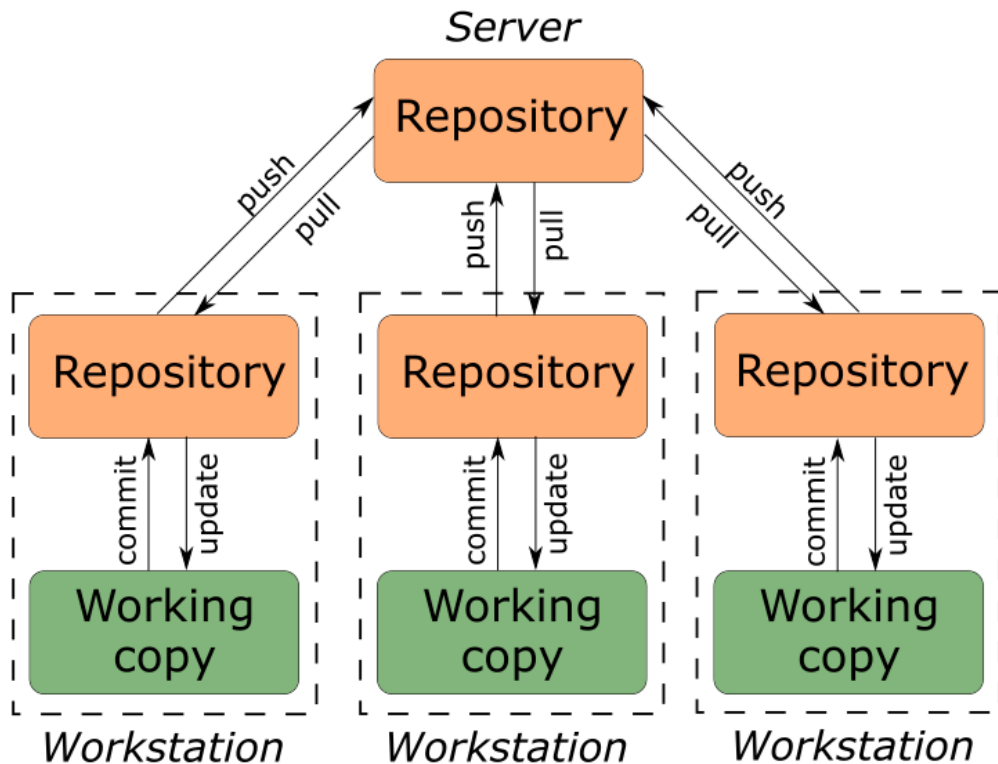
Непрерывная интеграция и доставка

ІІТМО



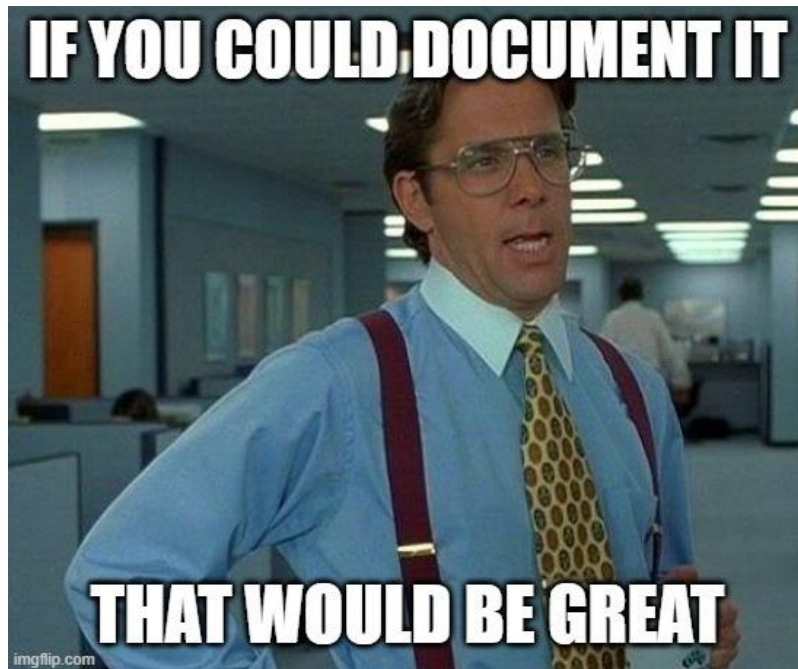
CI/CD — это технология автоматизации тестирования и доставки новых модулей разрабатываемого проекта заинтересованным сторонам (разработчикам, аналитикам, инженерам качества, конечным пользователям и др.).

Управление версиями



Система управления версиями — программное обеспечение для облегчения работы с изменяющейся информацией.

Инфраструктура как код



Инфраструктура как код (англ. **Infrastructure-as-Code**; Iac) — это подход для управления и описания инфраструктуры ЦОД через конфигурационные файлы, а не через ручное редактирование конфигураций на серверах или интерактивное взаимодействие

Сравним



СПЕЦИАЛЬНЫЙ
ТЕХНОЛОГИЧЕСКИЙ
ЦЕНТР

Как было раньше

```
File Edit View Bookmarks Settings Help
zynaddsubfx* test.sh*
ZynAddSubFX.lv2/ Timing1.plot
ZynAddSubFX.lv2presets/ Timing2.plot
ZynAddSubFX.so* Timings.html
zyn-fusion* token/
zyn-fusion-ELUA.txt trashy/
$ trashy.tar.bz2
travesty/
tt.sh*
vcard/
video-projects/
vids/
vmaf-1.3.9-x86_64-1_SMI.tgz*
wallpaper/
werkindjvu/
zyn-fusion-linux-64bit-3.0.2-patch4-release.tar.bz2
zyn-fusion-linux-64bit-3.0.2-release.tar.bz2
$

1 bash
ls*; export PERL_MM_OPT;
export JAVA_HOME=/usr/lib64/adoptopenjdk-11.0.6_10
export MANPATH=${MANPATH}:${JAVA_HOME}/man
export PATH=${PATH}:${JAVA_HOME}/bin:${JAVA_HOME}/jre/bin
%

3 bash
[beast] 0 bash 1 bash 2 bash 3 bash [04/30 3:16PM]
> sek : bash sek : screen
```

Как стало при IaC

```
inputs = {
  psql_service_name      = "delivery_service"
  psql_service_env       = "PRODUCTION"
  psql_service_disk_type = "network-ssd"
  psql_service_disk_size = "50"
  psql_db_names          = ["delivery_service"]
  psql_user              = "delivery_service"
}
```

- Самообслуживание (независимость от админов)
- Скорость и безопасность
- Документация
- Версионирование (все исходники можно хранить в системе контроля версий)
- Переиспользование
- Проверка перед выполнением действий



Популярные CSM

Ansible, Puppet, Chef, SaltStack

У каждого из них есть своя ниша и серебряной пули не существует

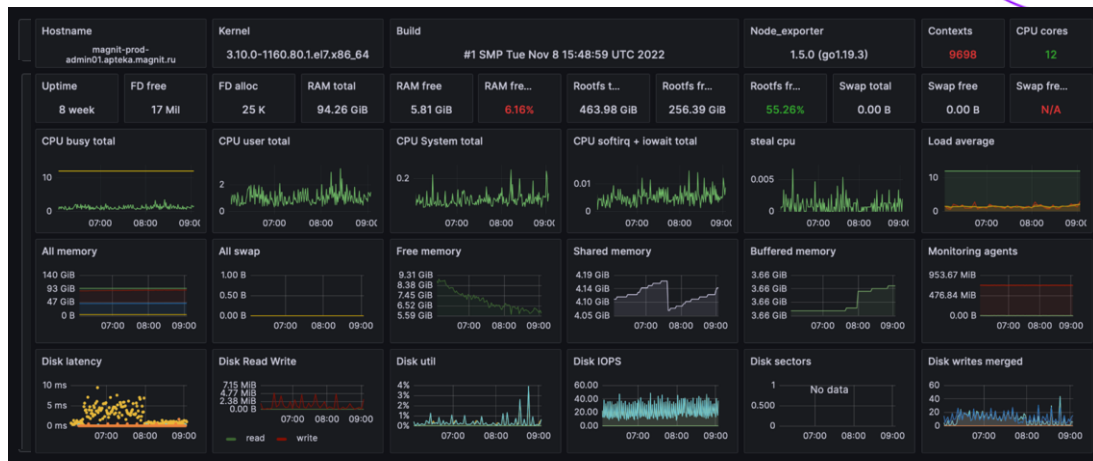
Puppet, Chef и SaltStack требуют развёртывания головного сервера и установки агентов на хостах. *Ansible* работает без агентов, он подключается к серверам по известным протоколам удаленного доступа (ssh, WinRM и тд)



Непрерывный мониторинг



Непрерывный мониторинг (Continuous Monitoring) - это сервис постоянного сканирования сетевого периметра, который позволяет обнаружить потенциально уязвимые сервисы и службы, а также оценить актуальные внешние угрозы и понять, кто из злоумышленников может вас атаковать.



Observability



The 3 pillars of Observability

Три кита observability:

- Метрики
- Логи
- Трейсы

Примеры

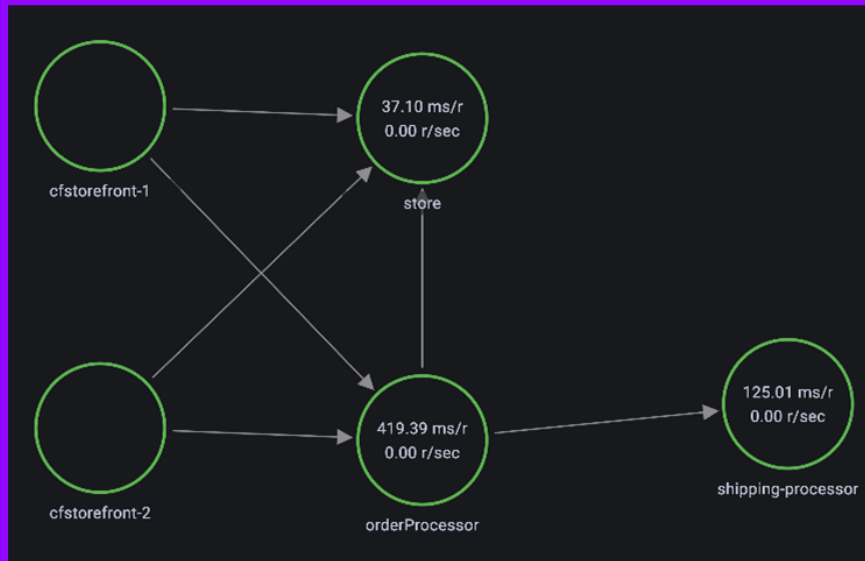
Metrics



Logs

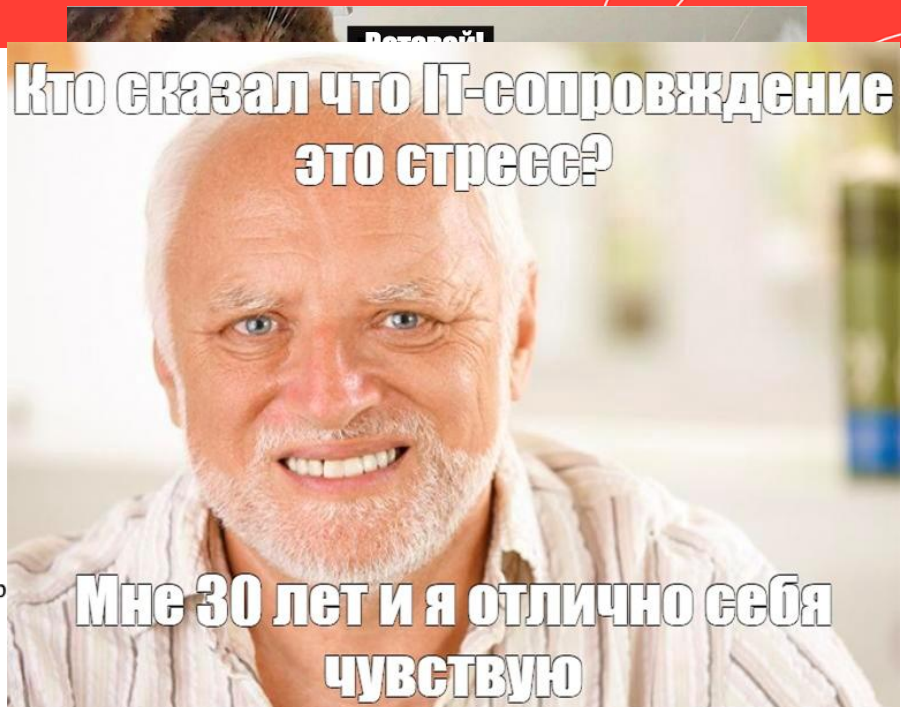
```
> 2023-08-27 09:14:54.278 {"level":"INFO","ts":"2023-08-27T06:14:54+0000","logger":"service.kafka_oms_order_saved","msg":["2023-08-27\n\\Symfony\\Component\\Messenger\\Event\\WorkerRunningEvent\\\" to listener \\Symfony\\Component\\Messenger\\E\n\\\", {\\\"event\\\":\\\"Symfony\\Component\\Messenger\\Event\\WorkerRunningEvent\\\", \\\"listener\\\":\\\"Symfony\nservicesListener::resetServices\\\"} ]}]\n\n> 2023-08-27 09:14:54.278 {"level":"INFO","ts":"2023-08-27T06:14:54+0000","logger":"service.kafka_oms_order_saved","msg":["2023-08-27\n\\Symfony\\Component\\Messenger\\Event\\WorkerRunningEvent\\\" to listener \\Symfony\\Component\\Messenger\\E\norkerRunning\\\", {\\\"event\\\":\\\"Symfony\\Component\\Messenger\\Event\\WorkerRunningEvent\\\", \\\"listener\\\":\\\"Symfony\ner\\StopWorkerOnRestartSignalListener::onWorkerRunning\\\"} ]}]\n\n2023-08-27T06:14:54.277988+00:00] event.DE\n\\Event\\WorkerRunningEvent\\\" to listener \\Symfony\\Component\\Messenger\\EventListener\\StopWorkerOnCustom\n\\\":\\\"Symfony\\Component\\Messenger\\Event\\WorkerRunningEvent\\\", \\\"listener\\\":\\\"Symfony\\Component\nStopExceptionListener::onWorkerRunning\\\"} ]}]\n\n2023-08-27T06:14:54.278015+00:00] event.DEBUG: Notified even\ningEvent\\\" to listener \\Symfony\\Component\\Messenger\\EventListener\\StopWorkerOnTimeLimitListener::onWor\nessenger\\Event\\WorkerRunningEvent\\\", \\\"listener\\\":\\\"Symfony\\Component\\Messenger\\EventListener\\
```

Tracing



Подведём итоги (было)

ИТМО



Кто сказал что IT-сопровождение
это стресс?

Мне 30 лет и я отлично себя
чувствую

- Множество виртуальных машин
- Настраиваем всё вручную
- Тратим дни на “а у меня локально всё работает”
- Разрабатываем по git flow
- Раз в 3 месяца деплоимся (по waterfall)
- Прод упал, мы теряем 500к/ннск, срочно почините (чиним 2 дня)
- Начинаем это всё заново

Подведём итоги (стало)

ИТМО



- Микросервисы в контейнерах (легко настраиваются и переносятся от разработчика до прода) — ✗
- Настраиваем всё через код
- Разрабатываем по TBD
- Деплой раз в день
- Что-то сломалось – нажал кнопку в CI/CD и вернул как было
- Поломку увидел по графикам, а не по разгневанным пользователям
- Работаешь DevOps-инженером

Но, эти подходы – не панацея

ИТМО



- История про Васю, и почему DevOps – не всегда хорошо



Цели и антипаттерны

- ✗ **"DevOps team"** - DevOps это культура, а не отдел!
- ✗ **100% uptime цель** - дорого и не нужно (error budget!)
- ✗ **Manual toil** – делаем руками
- ✗ **Blame culture** - боимся экспериментов
- ✗ **Игнорирование метрик** - не можешь измерить = не можешь улучшить
- ✗ **Hero culture** - один человек всё знает

- ✓ **Автоматизация** - если делаешь дважды, автоматизируй
- ✓ **Мониторинг всего** - нет алертов = нет проблем? Нет, просто не знаете о них
- ✓ **Инкрементальность** - маленькие изменения чаще
- ✓ **Fail fast** - лучше узнать о проблеме на CI, чем в production
- ✓ **Документация** - runbook для каждого алерта
- ✓ **Postmortems без обвинений** - ошибки - это обучение
- ✓ **Измеряй и улучшай** - DORA метрики

GitOps

- Декларативность (описываем ЧТО, а не КАК)
- Версионирование в Git
- Автоматическое применение
- Continuous reconciliation

Git Repo (desired state)



GitOps Controller (ArgoCD/Flux)



Kubernetes Cluster (actual state)

